

# MICROPROCESSOR & INTERFACING (BE05000551)

VERIFIED SOLUTIONS & DRY RUN REFERENCE MANUAL

CHALLENGE EXERCISES 1 TO 12

## EXPERIMENT NO. 1 • BASIC ARITHMETIC OPERATIONS

### Challenge Exercise: Sum with Multi-Location Carry Status Generation

**Problem Statement:** Write an 8085 assembly language program to add two 8-bit numbers. If a carry is generated, store 01H at memory location 2053H; otherwise, store 00H. Also, display the carry status by storing it in another designated memory location (2054H). Sum at 2052H.

```
LDA 2050H ; Load 1st operand MOV B,A ; Save in
reg B LDA 2051H ; Load 2nd operand ADD B ; A = A + B
STA 2052H ; Store Sum JC CARRY ; Jump if CY=1 MVI
A,00H ; No carry generated JMP STORE CARRY: MVI A,01H
; Carry generated STORE: STA 2053H ; Primary carry
flag loc STA 2054H ; Secondary designated loc HLT
```

#### I/O Allocation & Algorithm

- **Inputs:** 2050H = F5H, 2051H = 25H
  - **Sum Output:** 2052H = 1AH
  - **Status:** 2053H = 01H, 2054H = 01H
1. Load operands into A and B; execute ADD B.
  2. Branch via JC to assign flag indicator (01H vs 00H).
  3. Store flag status at both 2053H and 2054H.

| Step | Instruction | A   | B   | CY | Memory Updated | Operational Remarks                          |
|------|-------------|-----|-----|----|----------------|----------------------------------------------|
| 1    | LDA 2050H   | F5H | --  | 0  | --             | First operand loaded from 2050H              |
| 2    | MOV B,A     | F5H | F5H | 0  | --             | First operand cached into register B         |
| 3    | LDA 2051H   | 25H | F5H | 0  | --             | Second operand loaded into accumulator       |
| 4    | ADD B       | 1AH | F5H | 1  | --             | F5H + 25H = 11AH (Accumulator = 1AH, CY = 1) |
| 5    | STA 2052H   | 1AH | F5H | 1  | 2052H ← 1AH    | Sum byte stored at 2052H                     |
| 6    | JC CARRY    | 1AH | F5H | 1  | --             | Carry flag is 1; jump taken to CARRY label   |
| 7    | MVI A,01H   | 01H | F5H | 1  | --             | Accumulator loaded with carry status 01H     |
| 8    | STA 2053H   | 01H | F5H | 1  | 2053H ← 01H    | Primary carry byte written                   |
| 9    | STA 2054H   | 01H | F5H | 1  | 2054H ← 01H    | Secondary designated carry status updated    |
| 10   | HLT         | 01H | F5H | 1  | --             | Program execution terminated                 |

## EXPERIMENT NO. 2 • DECISION MAKING AND COMPARISON

### Challenge Exercise: Simultaneous Dual Comparison & Equality Audit

**Problem Statement:** Write an 8085 assembly language program to compare two 8-bit numbers. Store the larger number at 2052H, the smaller number at 2053H, and store 01H at 2054H if both numbers are equal; otherwise store 00H.

```
LDA 2050H MOV B,A LDA 2051H CMP B JZ EQUAL JC
LESS ; If A < B (2051H < 2050H) ; Case: 2051H > 2050H
STA 2052H ; Larger MOV A,B STA 2053H ; Smaller MVI
A,00H STA 2054H ; Equal = 00H HLT LESS: STA 2053H ;
Smaller MOV A,B STA 2052H ; Larger MVI A,00H STA 2054H
HLT EQUAL: STA 2052H STA 2053H MVI A,01H STA 2054H ;
Equal = 01H HLT
```

| Step | Instruction | A   | B   | Flags     | Memory Updated             |
|------|-------------|-----|-----|-----------|----------------------------|
| 1    | LDA 2050H   | 4CH | --  | --        | Input: 2050H = 4CH         |
| 2    | MOV B,A     | 4CH | 4CH | --        | Input: 2051H = 29H         |
| 3    | LDA 2051H   | 29H | 4CH | --        | Acc receives 2nd operand   |
| 4    | CMP B       | 29H | 4CH | CY=1, Z=0 | 29H - 4CH generates borrow |
| 5    | JZ EQUAL    | 29H | 4CH | CY=1, Z=0 | Not equal, branch skipped  |
| 6    | JC LESS     | 29H | 4CH | CY=1, Z=0 | Jump to LESS executed      |
| 7    | STA 2053H   | 29H | 4CH | CY=1, Z=0 | 2053H ← 29H (Smaller)      |
| 8    | MOV A,B     | 4CH | 4CH | CY=1, Z=0 | Load 4CH into Acc          |
| 9    | STA 2052H   | 4CH | 4CH | CY=1, Z=0 | 2052H ← 4CH (Larger)       |
| 10   | MVI A,00H   | 00H | 4CH | CY=1, Z=0 | Set equality flag byte     |
| 11   | STA 2054H   | 00H | 4CH | CY=1, Z=0 | 2054H ← 00H (Not Equal)    |

### EXPERIMENT NO. 3 • ARITHMETIC APPLICATIONS

Challenge Exercise: Expression Evaluation Result =  $(A \times B) + (C \div D)$

**Problem Statement:** Write an 8085 assembly language program to evaluate Result =  $(A \times B) + (C \div D)$ . Store result at 2054H using repeated addition for multiplication and repeated subtraction for division (robust for zero multipliers).

```
; Evaluation of (A * B) + (C / D) ; Mem:
2050H=A, 2051H=B, 2052H=C, 2053H=D, 2054H=Result LDA
2050H MOV B,A ; Multiplicand A LDA 2051H MOV C,A ;
Multiplier B MVI E,00H ; Product register E = 0 MULT:
MOV A,C ORA A ; Check if multiplier == 0 JZ DIV_START
MOV A,E ADD B MOV E,A DCR C JMP MULT DIV_START: LDA
2052H MOV B,A ; Dividend C LDA 2053H MOV C,A ; Divisor
D MVI D,00H ; Quotient register D = 0 DIV_L: MOV A,B
CMP C JC ADD_RES SUB C MOV B,A INR D JMP DIV_L
ADD_RES:MOV A,E ADD D ; A = (A * B) + (C / D) STA
2054H HLT
```

#### Sample Parameters & Verification

- **Inputs:** A=05H, B=04H → Product = 14H (20)
- **Inputs:** C=10H (16), D=03H (3) → Quotient = 05H (5)
- **Expected Result:** 14H + 05H = 19H (25 dec)

| Phase      | Key Regs       | Acc | Memory    | Action Taken           |
|------------|----------------|-----|-----------|------------------------|
| Mult Init  | B=05,<br>C=04  | 00H | E=00H     | Loop 4 times           |
| Mult End   | B=05,<br>C=00  | 14H | E=14H     | A*B product complete   |
| Div Step 1 | B=10,<br>C=03  | 0DH | D=01H     | 16 - 3 = 13 (0DH)      |
| Div Step 2 | B=0D,<br>C=03  | 0AH | D=02H     | 13 - 3 = 10 (0AH)      |
| Div Step 3 | B=0A,<br>C=03  | 07H | D=03H     | 10 - 3 = 7 (07H)       |
| Div Step 4 | B=07,<br>C=03  | 04H | D=04H     | 7 - 3 = 4 (04H)        |
| Div Step 5 | B=04,<br>C=03  | 01H | D=05H     | 4 - 3 = 1;<br>CMP→stop |
| Final Sum  | E=14H,<br>D=05 | 19H | 2054H←19H | 14H + 05H = 19H stored |

### EXPERIMENT NO. 4 • DATA PROCESSING AND ARRAY OPERATIONS

Challenge Exercise: Simultaneous Min & Max Array Traversal

**Problem Statement:** Write an 8085 assembly language program to find both the largest and smallest elements of an array in a single traversal, and store the results in separate memory locations (2060H = Smallest, 2061H = Largest).

```
LXI H,2050H ; HL points to array start MOV C,M
; Reg C = Current Smallest MOV D,M ; Reg D = Current
Largest MVI B,04H ; Counter = Remaining elements (5-1)
NEXT_EL:INX H MOV A,M ; Read element CMP C ; Compare
with Smallest JNC CHK_MAX ; If A >= C, skip Smallest
update MOV C,A ; Update Smallest JMP DEC_CNT
CHK_MAX:CMP D ; Compare with Largest JC DEC_CNT ; If A
< D, skip Largest update MOV D,A ; Update Largest
DEC_CNT:DCR B JNZ NEXT_EL MOV A,C STA 2060H ; Store
Smallest at 2060H MOV A,D STA 2061H ; Store Largest at
2061H HLT
```

| Iter  | Addr  | Data | Smallest (C) | Largest (D) | Counter (B) |
|-------|-------|------|--------------|-------------|-------------|
| Init  | 2050H | 25H  | 25H          | 25H         | 04H         |
| 1     | 2051H | 68H  | 25H          | 68H         | 03H         |
| 2     | 2052H | 19H  | 19H          | 68H         | 02H         |
| 3     | 2053H | 42H  | 19H          | 68H         | 01H         |
| 4     | 2054H | 55H  | 19H          | 68H         | 00H         |
| Store | 2060H | --   | 19H          | --          | 2060H ← 19H |
| Store | 2061H | --   | --           | 68H         | 2061H ← 68H |

*Audit Note:* Evaluates upper and lower extrema concurrently within a single  $O(N)$  sweep through pointer auto-incrementing.

## EXPERIMENT NO. 5 • LOGICAL AND BIT MANIPULATION

### Challenge Exercise: Deterministic Bit Analysis & Parity Generator

**Problem Statement:** Write an 8085 assembly language program to count the number of 1's and 0's in an 8-bit number and determine whether the number has even parity or odd parity. Store 1's count at 2051H, 0's count at 2052H, and parity (00H=Even, 01H=Odd) at 2053H.

```
LDA 2050H ; Load input operand MVI B,08H ; 8-bit loop counter MVI C,00H ; Count of 1's MVI D,00H ; Count of 0's BIT_LP: RRC ; Rotate bit D0 into CY deterministically JC BIT_ONE INR D ; Increment 0's counter JMP CHK_END BIT_ONE:INR C ; Increment 1's counter CHK_END:DCR B JNZ BIT_LP MOV A,C STA 2051H ; 2051H = Total 1's MOV A,D STA 2052H ; 2052H = Total 0's MOV A,C ANI 01H ; Test LSB of 1's count JZ EVEN_P MVI A,01H ; Odd parity indicator JMP ST_PAR EVEN_P: MVI A,00H ; Even parity indicator ST_PAR: STA 2053H HLT
```

#### Dry Run Trace (Input: 2050H = A5H = 10100101B)

| Bit Step | Shifted Bit | CY | C (1s) | D (0s) | B (Rem) |
|----------|-------------|----|--------|--------|---------|
| 1 (D0)   | 1           | 1  | 01H    | 00H    | 07H     |
| 2 (D1)   | 0           | 0  | 01H    | 01H    | 06H     |
| 3 (D2)   | 1           | 1  | 02H    | 01H    | 05H     |
| 4 (D3)   | 0           | 0  | 02H    | 02H    | 04H     |
| 5 (D4)   | 0           | 0  | 02H    | 03H    | 03H     |
| 6 (D5)   | 1           | 1  | 03H    | 03H    | 02H     |
| 7 (D6)   | 0           | 0  | 03H    | 04H    | 01H     |
| 8 (D7)   | 1           | 1  | 04H    | 04H    | 00H     |

Parity Test: C = 04H → 04H AND 01H = 00H → Even Parity → 2053H = 00H.

## EXPERIMENT NO. 6 • LOOPING, DELAY AND COUNTER PROGRAMS

### Challenge Exercise: 10 Square-Wave Pulses with Pulse Tracking Counter

**Problem Statement:** Write an 8085 assembly language program to generate a sequence of 10 square-wave pulses with equal ON and OFF durations using software delay loops, initializing the stack pointer, and maintain a pulse counter at 2051H (waveform on 2050H).

```
LXI SP,30FFH ; Initialize Stack Pointer MVI C,0AH ; 10 pulse counter MVI D,00H ; Completed pulse register PULSE: MVI A,FFH ; Logic HIGH STA 2050H ; Output port/memory CALL DELAY ; Half-period delay MVI A,00H ; Logic LOW STA 2050H CALL DELAY ; Half-period delay INR D ; Pulse completed MOV A,D STA 2051H ; Update pulse tally DCR C ; Decrement burst counter JNZ PULSE HLT DELAY: MVI B,10H ; Calibrated inner count DEL_LP: DCR B JNZ DEL_LP RET
```

| Pulse # | 2050H (Level) | Delay Routine | 2051H (Counter) | Remaining (C)    |
|---------|---------------|---------------|-----------------|------------------|
| 1       | FFH → 00H     | Executed (2x) | 01H             | 09H              |
| 2       | FFH → 00H     | Executed (2x) | 02H             | 08H              |
| 3       | FFH → 00H     | Executed (2x) | 03H             | 07H              |
| 4       | FFH → 00H     | Executed (2x) | 04H             | 06H              |
| 5       | FFH → 00H     | Executed (2x) | 05H             | 05H              |
| ...     | FFH → 00H     | Executed (2x) | ...             | ...              |
| 10      | FFH → 00H     | Executed (2x) | 0AH             | 00H (Terminates) |

*Result:* Generates exactly 10 clean square wave cycles while persistently logging the completed cycle count in real time.

## EXPERIMENT NO. 7 • STACK AND SUBROUTINE PROGRAMMING

### Challenge Exercise: Expression (A + B) – (C + D) with Stack Preservation

**Problem Statement:** Write an 8085 assembly language program to calculate the expression (A + B) – (C + D) using two separate subroutines. Store the final result in memory (2054H) and use stack operations to preserve register contents during subroutine execution.

```
LXI SP,30FFH ; Initialize Stack Pointer
LDA 2050H ; A operand
MOV B,A
LDA 2051H ; B operand
CALL ADD_SUB ; Calculate (A + B)
MOV D,A ; D = (A + B)
LDA 2052H ; C operand
MOV B,A
LDA 2053H ; D operand
CALL ADD_SUB ; Calculate (C + D)
MOV E,A ; E = (C + D)
CALL SUB_SUB ; Calculate D - E
STA 2054H ; Store Final Difference
HLT ; Subroutine: Addition with Register Preservation
ADD_SUB: PUSH B ; Preserve Register B on Stack
ADD B
POP B ; Restore Register B
RET ; Subroutine: Subtraction with Register Preservation
SUB_SUB: PUSH B
MOV A,D
SUB E
POP B
RET
```

#### Dry Run Trace (Inputs: A=20H, B=15H, C=10H, D=05H)

| Stage  | Instruction  | SP    | Stack Top | Acc | Remarks                |
|--------|--------------|-------|-----------|-----|------------------------|
| Init   | LXI SP,30FFH | 30FFH | --        | --  | Stack initialized      |
| Call 1 | CALL ADD_SUB | 30FDH | Ret Addr  | 15H | Pushes return address  |
| Push B | PUSH B       | 30FBH | 20xxH     | 15H | B cached on stack      |
| Exec   | ADD B & POP  | 30FDH | Ret Addr  | 35H | (A+B)=35H, B popped    |
| Call 2 | CALL ADD_SUB | 30FDH | Ret Addr  | 05H | Pushes return address  |
| Exec   | ADD B & POP  | 30FDH | Ret Addr  | 15H | (C+D)=15H, B popped    |
| Call 3 | CALL SUB_SUB | 30FDH | Ret Addr  | --  | Enter subtract routine |
| Exec   | SUB E & RET  | 30FFH | --        | 20H | 35H - 15H = 20H        |
| Done   | STA 2054H    | 30FFH | --        | 20H | 2054H ← 20H            |

## EXPERIMENT NO. 8 • CODE CONVERSION AND BCD ARITHMETIC

### Challenge Exercise: BCD → Binary → Addition → BCD Conversion Pipeline

**Problem Statement:** Write an 8085 assembly language program to accept a two-digit BCD number (2050H), convert it into Binary, add it to another Binary number (2051H), and convert the final result back into BCD before storing it in memory (2052H).

```
; Input: 2050H=25H (BCD 25), 2051H=08H (Binary 8)
LDA 2050H ; Packed BCD byte
MOV B,A ; Keep copy
ANI 0FH ; Extract Lower Nibble
MOV C,A ; C = Units digit
MOV A,B
ANI F0H ; Extract Tens Nibble
RRC
RRC
RRC
RRC ; A = Tens digit
MOV D,A ; D = Tens multiplier count
MVI E,0AH ; Constant 10
MVI A,00H ; Accumulator for multiplication
MOV H,D
ORA H ; Guard: check if Tens == 0
JZ ADD_UNITS ; Skip multiplication if Tens is 0
CONV: ADD E ; Compute Tens * 10
DCR D
JNZ CONV
ADD_UNITS: ADD C ; A = (Tens*10) + Units = Binary
MOV B,A ; Save Binary equivalent
LDA 2051H ; Load binary addend
ADD B ; A = Bin1 + Bin2 ; Convert back to BCD via repeated subtraction of 10
MVI C,00H ; BCD Tens counter
BCD_LP: CMP E ; Compare with 10
JC BCD_FIN
SUB E
INR C
JMP BCD_LP
BCD_FIN: MOV D,A ; D = Remainder
Units: MOV A,C ; Tens
RLC
RLC
RLC
RLC ; Shift tens to upper nibble
ORA D ; Combine tens and units
STA 2052H ; Store Packed BCD result
HLT
```

| Operation Stage  | Data | Acc | Regs     | Interpretation         |
|------------------|------|-----|----------|------------------------|
| Read BCD         | 25H  | 25H | B=25H    | Packed BCD loaded      |
| Units Extract    | 05H  | 05H | C=05H    | Lower nibble isolated  |
| Tens Extract     | 20H  | 02H | D=02H    | Upper digit isolated   |
| Tens × 10        | --   | 14H | A=20 dec | 02H × 0AH = 14H (20)   |
| BCD → Bin Result | --   | 19H | B=19H    | 20 + 5 = 25 (19H)      |
| Add Binary       | 08H  | 21H | A=21H    | 25 + 8 = 33 (21H)      |
| Bin → BCD Tens   | --   | 03H | C=03H    | 21H(33) / 10 = 3       |
| Pack BCD         | --   | 33H | D=03H    | (03H × 16)   03H = 33H |
| Output           | 33H  | 33H | --       | 2052H ← 33H (BCD)      |

## EXPERIMENT NO. 9 • ADVANCED ARITHMETIC AND MATHEMATICAL

### Challenge Exercise: Arithmetic Engine ( $A \times B$ ) + ( $C \times D$ ) with 16-Bit Precision

**Problem Statement:** Write an 8085 assembly language program to evaluate the composite expression  $(A \times B) + (C \times D)$  using repeated addition multiplication routines and store the result across memory locations 2054H–2055H.

```
; Evaluation of (A * B) + (C * D) ; Mem:
2050H=A, 2051H=B, 2052H=C, 2053H=D, 2054H-55H=Sum LDA
2050H MOV B,A ; Multiplicand A LDA 2051H MOV C,A ;
Multiplier B LXI H,0000H ; Clear Product 1 M1_LP: MOV
A,C ORA A JZ NEXT_M MOV E,B MVI D,00H DAD D ; HL = HL
+ A DCR C JMP M1_LP NEXT_M: SHLD 2054H ; Store first
product LDA 2052H MOV B,A ; Multiplicand C LDA 2053H
MOV C,A ; Multiplier D LXI H,0000H ; Clear Product 2
M2_LP: MOV A,C ORA A JZ SUM_UP MOV E,B MVI D,00H DAD D
; HL = HL + C DCR C JMP M2_LP SUM_UP: XCHG ; DE = (C *
D) LHLD 2054H ; HL = (A * B) DAD D ; HL = (A * B) + (C
* D) SHLD 2054H ; Store composite sum HLT
```

Trace (A=0AH [10], B=0CH [12], C=08H [8], D=0FH [15])

| Computational Step | HL Reg | DE Reg | Product / Accumulation       |
|--------------------|--------|--------|------------------------------|
| P1 Init            | 0000H  | 000AH  | B=0AH, C=0CH (12 additions)  |
| P1 Complete        | 0078H  | 000AH  | $10 \times 12 = 120$ (0078H) |
| Cache P1           | 0078H  | --     | 2054H←78H, 2055H←00H         |
| P2 Init            | 0000H  | 0008H  | B=08H, C=0FH (15 additions)  |
| P2 Complete        | 0078H  | 0008H  | $8 \times 15 = 120$ (0078H)  |
| Register Swap      | --     | 0078H  | DE holds Product 2           |
| Reload P1          | 0078H  | 0078H  | HL holds Product 1           |
| DAD D Execution    | 00F0H  | 0078H  | $120 + 120 = 240$ (00F0H)    |
| Final Write        | 00F0H  | 0078H  | 2054H ← F0H, 2055H ← 00H     |

## EXPERIMENT NO. 10 • MEMORY AND INPUT/OUTPUT PROGRAMMING

### Challenge Exercise: Synchronous Block I/O Transfer via Memory Buffer

**Problem Statement:** Write an 8085 assembly language program to read 10 bytes from an input port (01H), store them sequentially into a memory buffer (2050H–2059H), and subsequently transmit the stored buffer sequentially to an output port (02H).

```
; Phase 1: Input Port to Memory Buffer LXI
H,2050H ; Buffer base address MVI B,0AH ; 10 bytes
transfer count IN_LP: IN 01H ; Read from peripheral
port 01H MOV M,A ; Store byte in memory buffer INX H ;
Advance memory pointer DCR B JNZ IN_LP ; Phase 2:
Memory Buffer to Output Port LXI H,2050H ; Reset
pointer to buffer start MVI B,0AH ; Reset block
counter OUT_LP: MOV A,M ; Read byte from buffer OUT
02H ; Send byte to output port 02H INX H ; Advance
pointer DCR B JNZ OUT_LP HLT
```

| Byte | Input (01H) | Buffer Addr | Buffered | Output (02H) | Counter B        |
|------|-------------|-------------|----------|--------------|------------------|
| 1    | 11H         | 2050H       | 11H      | 11H          | 09H              |
| 2    | 22H         | 2051H       | 22H      | 22H          | 08H              |
| 3    | 33H         | 2052H       | 33H      | 33H          | 07H              |
| 4    | 44H         | 2053H       | 44H      | 44H          | 06H              |
| ...  | ...         | ...         | ...      | ...          | ...              |
| 9    | 99H         | 2058H       | 99H      | 99H          | 01H              |
| 10   | AAH         | 2059H       | AAH      | AAH          | 00H (Terminates) |

*Architecture Note:* Decouples input acquisition from transmission, preventing data loss during peripheral rate mismatches.

## EXPERIMENT NO. 11 • APPLICATION-ORIENTED PROGRAMMING

### Challenge Exercise: Multi-Sensor Supervisory Monitor with Alarm Tripping

**Problem Statement:** Write an 8085 assembly language program to monitor five sensor channels stored from 2050H onwards against predefined critical thresholds stored from 2060H onwards. If any sensor value exceeds its threshold, activate an alarm by writing FFH to Output Port 01H and log breach count at 2070H; otherwise output 00H.

```
LXI H,2050H ; HL -> Sensor readings LXI
D,2060H ; DE -> Threshold benchmarks MVI B,05H ;
Number of sensors = 5 MVI C,00H ; Breach counter SCAN:
LDAX D ; A = Threshold value CMP M ; CMP M computes (A
- M) = (Threshold - Sensor) JNC OK ; If Threshold >=
Sensor, CY=0 (Normal) INR C ; If Sensor > Threshold,
CY=1 (Breach!) OK: INX H INX D DCR B JNZ SCAN MOV A,C
STA 2070H ; Store total breaches ORA A ; Test breach
tally JZ NO_ALARM MVI A,FFH ; Trip alarm code OUT 01H
JMP HALT_PRG NO_ALARM:MVI A,00H ; Normal operation
code OUT 01H HALT_PRG:HLT
```

| Channel | Sensor (M)                      | Limit (DE) | Status | Breach (C) | Port 01H |
|---------|---------------------------------|------------|--------|------------|----------|
| CH1     | 35H (53°C)                      | 40H (64°C) | Normal | 00H        | --       |
| CH2     | 52H (82°C)                      | 45H (69°C) | BREACH | 01H        | --       |
| CH3     | 18H (24°C)                      | 30H (48°C) | Normal | 01H        | --       |
| CH4     | 61H (97°C)                      | 50H (80°C) | BREACH | 02H        | --       |
| CH5     | 22H (34°C)                      | 25H (37°C) | Normal | 02H        | --       |
| Summary | 2 Breaches Logged → 2070H = 02H |            |        | 02H        | FFH      |

## EXPERIMENT NO. 12 • ADVANCED PROGRAMMING CHALLENGES & INTEGRATED APPLICATIONS

### Integrated Challenge: Student Result Processing & Inventory Management

**Problem Statements:** (1) Process marks of 10 students (2050H–2059H): calculate Total, Highest, Lowest, Pass ( $\geq 35$  [23H]), and Fail counts. (2) Inventory System: compare 10 items against threshold 0AH, log 01H (restock) / 00H (OK) at 2060H–2069H, and total low stock count at 2070H.

Program 1: Student Result Processing Dry Run Table (Pass Mark  $\geq 23$ H [35])

| Student | Address | Marks    | Running Total | Highest (B) | Lowest (C) | Result (Pass/Fail) |
|---------|---------|----------|---------------|-------------|------------|--------------------|
| S1      | 2050H   | 48H (72) | 48H           | 48H         | 48H        | Pass (P=1, F=0)    |
| S2      | 2051H   | 3CH (60) | 84H           | 48H         | 3CH        | Pass (P=2, F=0)    |
| S3      | 2052H   | 21H (33) | A5H           | 48H         | 21H        | FAIL (P=2, F=1)    |
| S4      | 2053H   | 55H (85) | FAH           | 55H         | 21H        | Pass (P=3, F=1)    |
| S5      | 2054H   | 2DH (45) | 27H (CY=1)    | 55H         | 21H        | Pass (P=4, F=1)    |
| S6      | 2055H   | 1EH (30) | 45H           | 55H         | 1EH        | FAIL (P=4, F=2)    |
| S7      | 2056H   | 41H (65) | 86H           | 55H         | 1EH        | Pass (P=5, F=2)    |
| S8      | 2057H   | 32H (50) | B8H           | 55H         | 1EH        | Pass (P=6, F=2)    |
| S9      | 2058H   | 5AH (90) | 12H (CY=1)    | 5AH         | 1EH        | Pass (P=7, F=2)    |
| S10     | 2059H   | 24H (36) | 36H           | 5AH         | 1EH        | Pass (P=8, F=2)    |

Final Report: 2060H=36H (Total LSB), 2061H=5AH (High), 2062H=1EH (Low), 2063H=08H (Pass), 2064H=02H (Fail)

Program 2: Inventory Management System Dry Run Table (Threshold = 0AH [10 Units])

| Product | Source Addr | Quantity | Threshold | Status Stored (2060H–2069H)      | Low-Stock Count (2070H) |
|---------|-------------|----------|-----------|----------------------------------|-------------------------|
| P1      | 2050H       | 15H (21) | 0AH (10)  | 2060H ← 00H (Stock Sufficient)   | 00H                     |
| P2      | 2051H       | 08H (8)  | 0AH (10)  | 2061H ← 01H (Replenish Required) | 01H                     |
| P3      | 2052H       | 0CH (12) | 0AH (10)  | 2062H ← 00H (Stock Sufficient)   | 01H                     |
| P4      | 2053H       | 05H (5)  | 0AH (10)  | 2063H ← 01H (Replenish Required) | 02H                     |
| P5      | 2054H       | 12H (18) | 0AH (10)  | 2064H ← 00H (Stock Sufficient)   | 02H                     |
| P6      | 2055H       | 09H (9)  | 0AH (10)  | 2065H ← 01H (Replenish Required) | 03H                     |
| P7      | 2056H       | 14H (20) | 0AH (10)  | 2066H ← 00H (Stock Sufficient)   | 03H                     |
| P8      | 2057H       | 07H (7)  | 0AH (10)  | 2067H ← 01H (Replenish Required) | 04H                     |
| P9      | 2058H       | 11H (17) | 0AH (10)  | 2068H ← 00H (Stock Sufficient)   | 04H                     |
| P10     | 2059H       | 06H (6)  | 0AH (10)  | 2069H ← 01H (Replenish Required) | 05H                     |

Final Report: 2060H–2069H contains Status Array; 2070H = 05H (Total Low-Stock Products)